

Hierarchical Insider Threat Detection: Integrating Lightweight Screening with LLM-Based Reasoning and Graph-Informed Narratives

Ziyuan Liu
University of Ottawa
Ottawa, Canada
zliu053@uottawa.ca

Jingxuan Xu
University of Ottawa
Ottawa, Canada
jxu022@uottawa.ca

Kair Wang
University of Ottawa
Ottawa, Canada
bwang105@uottawa.ca

Sabrina Cai
University of Ottawa
Ottawa, Canada
hcai062@uottawa.ca

Abstract—Insider threat detection is difficult for a practical reason that every security team recognizes immediately: malicious behavior is rare, evidence is scattered across several enterprise log sources, and reviewing every user-day manually is impossible [2]. This project studies that problem on the CERT r4.2 benchmark and asks a concrete question: can we preserve high recall while sharply reducing analyst workload? We design a two-stage pipeline. Stage 1 performs lightweight screening over user-day windows using 17 structured features from logon, file, device, and email logs. Stage 2 turns only the most suspicious windows into *graph-informed narratives* that summarize entities, temporal relations, recent history, LDAP context, and cross-source activity, and then scores those narratives with rule, TF-IDF, LLM, and fusion models.

The main empirical finding is that the hierarchy works. On the user-level held-out test split, the Stage 1 XGBoost screener attains precision 0.1037, recall 0.4863, F1 0.1710, and PR-AUC 0.2837. When all user-day windows are scored and globally ranked, the top 3% form a Stage 2 candidate pool of 9,908 windows containing 89.74% of all malicious windows. On the aligned Stage 2 evaluation split, TF-IDF with logistic regression is the strongest single second-stage model (F1 0.450, PR-AUC 0.365); the best reported fusion raises recall to 0.810 at F1 0.432. The LLM alone does not beat TF-IDF on this split, reflecting the fact that partially templated narratives expose repeated lexical cues that a lightweight statistical model exploits effectively. Fusion is therefore the more informative result: LLM scores are more useful here as a complementary signal than as a standalone replacement for strong non-LLM baselines.

Index Terms—Insider threat detection, hierarchical detection pipeline, Large Language Models (LLM), TF-IDF.

I. INTRODUCTION

Insider threats remain one of the most costly and conceptually difficult cybersecurity problems because the attacker already has legitimate access. That single fact changes everything. External attackers often look obviously foreign to a network; insiders do not. Their actions may be harmful, but they are usually built from ordinary enterprise behaviors such as logging in, reading files, sending emails, plugging in removable devices, or browsing the web. The challenge is not merely to spot anomalous events. It is to detect suspicious *combinations* of events, often spread across time and data sources, without overwhelming analysts with false alarms.

This project addresses the following problem: given multi-source enterprise audit data, identify malicious *user-day windows* with high recall while keeping the number of windows that require deeper inspection small enough to be operationally manageable. This formulation is important for three reasons. First, it reflects how many enterprise security workflows are organized in practice: analysts often investigate behavior over a day or other bounded review window rather than in isolated event records. Second, it aligns with the CERT scenario design, where insider behavior unfolds through sequences of actions rather than one-off outliers. Third, it forces the study to confront the central operational trade-off: missing dangerous cases is unacceptable, but escalating every mildly suspicious user-day is equally unrealistic.

Prior work has already shown that insider threat detection requires combining behavior, context, and anomaly signals [3], [4], [6], but no single family of methods solves the whole problem cleanly. Traditional machine learning models are scalable and effective on structured features, yet they compress context aggressively. Graph-based approaches capture relationships but can be heavy to build, tune, and explain. Deep models and recent LLM-based systems promise richer reasoning, but using them directly on full enterprise logs is expensive and difficult to calibrate [9]. The contribution of this study is therefore not a claim that one model class supersedes the rest. Instead, the study proposes a hierarchical design that uses each method where it is strongest: a structured screener to reduce scale, then a richer narrative representation for second-stage discrimination.

The paper makes three focused contributions. First, it formulates insider detection as a hierarchical ranking-and-refinement problem over user-day windows rather than as a flat event classifier [8]. Second, it introduces a graph-informed narrative representation that preserves cross-source context in readable text without requiring a separate graph-learning stack. Third, it evaluates the resulting pipeline against strong second-stage baselines—rules, TF-IDF, LLM prompting, and fusion—so that the empirical discussion can distinguish genuinely useful components from merely fashionable ones.

II. RELATED WORK AND STATE OF THE ART

The modern insider-threat literature makes two themes clear. First, the problem is intrinsically heterogeneous: malicious behavior is embedded in ordinary work activity, spans different data types, and is extremely imbalanced. Second, progress has repeatedly come from better representations of behavior rather than from a single universally superior classifier. Systematic reviews by Gheyas and Abdallah [1], Homoliak et al. [2], and Yuan and Wu [3] collectively show how the field evolved from rule-based and statistical anomaly detectors toward multi-domain, graph-based, and deep learning approaches. These reviews also highlight recurring obstacles that remain visible in this project: scarcity of realistic labels, heterogeneity across log sources, severe class imbalance, and the difficulty of comparing methods fairly across different data generation and evaluation protocols.

Early operational work emphasized relational structure and cross-domain fusion. Eberle and Holder [4] explored graph-based anomaly detection for insider activity, arguing that relationships between users, resources, and events can reveal threats that isolated features miss. Eldardiry et al. [5] pushed the field toward multi-domain information fusion, showing that behaviors that appear ordinary within any one domain may become suspicious when viewed jointly across logon, email, device, and file access data. This observation is central to the present project: an apparently harmless file event, USB event, or external email can look very different once placed into a shared temporal context.

Later work expanded this idea with stronger temporal and representation learning components. Tuor et al. [6] applied deep learning to structured cybersecurity streams and showed that unsupervised neural models could provide useful anomaly scores while reducing analyst burden. Hong et al. [8] developed a daily-activity graph framework, further reinforcing the usefulness of user-day representations and relational structure. At the same time, the survey by Yuan and Wu [3] made a careful point that still matters here: deep models can help, but class imbalance, lack of labels, and evaluation realism remain unresolved bottlenecks.

An equally important branch of the literature concerns the benchmark data itself. Glasser and Lindauer [7] explained the motivation, strengths, and limitations of synthetic insider-threat data generation, including the trade-off between experimental accessibility and ecological realism. This matters because the CERT family has become a standard experimental platform precisely because real insider datasets are difficult to share, but it also means any conclusions must be interpreted with appropriate caution.

Most recently, LLMs have entered cyber threat detection research. Chen et al. [9] survey the broader use of LLMs in threat detection tasks, especially where text, explanation, and reasoning are central. Song et al. [10] go further by applying LLM fine-tuning directly to insider-threat behavior logs. These newer studies suggest a compelling direction: language models may help when the detection problem can be reformulated

as interpretable text. However, this promise raises a practical question that the present project treats seriously: must one choose between scalable tabular learning and richer narrative reasoning? The answer proposed here is no. This study fits in the literature as a bridge between multi-domain feature engineering and language-based semantic analysis. It does not replace structured screening with an LLM, nor does it stop at hand-crafted tabular features. Instead, it uses structured screening to make richer second-stage reasoning affordable.

III. PROPOSED METHOD

A. Problem formulation

Let u denote a user and t denote a calendar day. The project defines one sample as a user-day window (u, t) . For each window, the preprocessing pipeline constructs a structured feature vector $\mathbf{x}_{u,t}$ from enterprise logs and a binary label $y_{u,t} \in \{0, 1\}$ indicating whether the window overlaps an official malicious period in the ground-truth insider file. The overall task is to learn a ranking-and-classification procedure that maximizes malicious-window recall while minimizing the number of windows sent to expensive downstream review.

The system is hierarchical. Stage 1 produces a screening score

$$s_{u,t}^{(1)} = f_{\theta}(\mathbf{x}_{u,t}),$$

where f_{θ} is a structured model such as XGBoost or Random Forest. Only the top- K windows by $s^{(1)}$ are retained:

$$\Omega_K = \text{TopK}(\{s_{u,t}^{(1)}\}).$$

For each retained window, Stage 2 creates a narrative representation $n_{u,t}$ and computes a second-stage score

$$s_{u,t}^{(2)} = g_{\phi}(n_{u,t}),$$

where g_{ϕ} can be a rule function, a TF-IDF classifier, an LLM, or a calibrated fusion model. This design answers the practical operational question of which windows justify scarce analyst time or expensive semantic reasoning.

B. Stage 1: lightweight screening

Stage 1 is deliberately simple, fast, and recall-oriented. The pipeline first aggregates raw events into per-user, per-day counts and burst ratios, then fits cost-sensitive classifiers to those features. The repository compares XGBoost and Random Forest because both are strong, interpretable tabular baselines and both naturally output continuous risk scores suitable for ranking. The Stage 1 screener is not treated as a final judge of maliciousness; its purpose is to retain as many truly malicious windows as possible in a dramatically smaller candidate pool.

This perspective changes how the model is evaluated. Top- K metrics are reported alongside precision, recall, F1, and PR-AUC so that the screener is judged both on classification quality at a fixed decision boundary and on its operational efficiency in isolating malicious instances within a constrained candidate set. In this project, $K = 3\%$ is an explicit budget choice rather than a structurally unique elbow, because Stage 2 narrative-generation and LLM-evaluation cost scale

approximately linearly with K . The repository therefore reports a Top- K sweep over $\{5\%, 3\%, 2\%\}$ under a fixed protocol, trading Recall@Pool against pool size: 2% is cheaper but loses noticeably more malicious windows, 5% attains higher coverage at the cost of forwarding more benign windows into the expensive second stage, and 3% yields a pool of roughly ten thousand windows while preserving about 89.7% XGBoost Recall@Pool. The methodological point is that K is one operating point on a recall–cost curve; in deployment, it should be re-swept to match SOC throughput and false-positive tolerance.

C. Stage 2: graph-informed narrative analysis

Once Stage 1 identifies suspicious windows, Stage 2 turns each one into a narrative designed for richer downstream reasoning. The key idea is not to store a formal graph database and then query it. Instead, the system extracts graph-like structure and verbalizes it. Each narrative identifies entities such as user, host, files, removable-device events, email recipients, and URLs, then organizes those entities by temporal and semantic relations.

Each narrative has four components:

- **Window header and user context.** The header summarizes the user and day. When LDAP information is available, the narrative also includes role and department.
- **Key same-day events with a cross-department cue.** The system lists the most salient events and annotates obvious conflicts, such as accesses whose resource type appears inconsistent with the user’s department. The conflict cue is a lightweight rule, not a learned classifier. The department string is retrieved from LDAP at the window date, and the accessed resource is assigned a coarse type from file-name or path substrings (`financial`, `hr`, `highly_sensitive`, `code`, `general`). A conflict is emitted only when the resource is typed as `financial` and the department does not contain “`financ`,” or when it is typed as `hr` and the department contains neither “`human`” nor “`hr`.” Resources typed as `general` never trigger the rule. The rule is an interpretable cue for downstream models; like any heuristic, it can miss true conflicts or flag benign cases.
- **Historical comparison.** The system compares current behavior to the continuous natural-day interval $[t-30 \text{ days}, t)$ for a window on day t , excluding the current day and filtering events by timestamp. The historical block is used as narrative context rather than as a numeric detector, so it does not rely on ratio denominators and has no sparse-history division issues. When the lookback is empty for an activity type, the narrative states this explicitly (e.g., “no file-related activity was observed”). The lookback is not truncated by employment start date, so a newly onboarded user and a long-tenured user with no recent logs may produce similar text—a semantic limitation of the narrative layer rather than a numerical bug.

- **Cross-source chronology.** Device, email, and HTTP events are ordered into a compact timeline so that multi-step chains—such as sensitive file access followed by USB use or external email—become readable.

The term “graph-informed” therefore denotes that the system extracts structural nodes and relations but renders them as grounded natural language. This approach is lighter than a full graph-learning pipeline, yet it preserves multi-entity context in a format accessible to both human analysts and language models.

D. Stage 2 scoring models

The project evaluates five second-stage scorers on the same narrative representation.

Stage 1 pool baseline. This baseline escalates every window that survived Stage 1 and has no Stage 2 intelligence at all. Its purpose is to measure how much value the second stage adds beyond the shortlist itself.

Rule baseline. The released implementation scores each narrative by summing fixed positive weights for a bank of regular-expression patterns over the English narrative text, then clipping the total at 1.0. Beyond after-hours and external-email/exfiltration cues, the pattern bank includes lightweight indicators for cross-department wording, USB and removable-media mentions, generic “unusual,” “first time,” “spike,” or “vs user avg” phrasing, abnormal or large attachments, HTTP/URL activity, termination or departure language, and encryption or archive references near sensitive paths. The weights are chosen for interpretability rather than tuned on training data. The exact pattern list is part of the released artifact `stage2_baselines.py`.

TF-IDF + Logistic Regression. This baseline converts narratives into unigram and bigram TF-IDF vectors (vocabulary cap 8,000, `min_df=2`, English stop-word removal) and fits a balanced logistic regression classifier (`max_iter=200`). Removing stop words is conservative for cybersecurity text, where common words can still carry context, but it reduces the dominance of ultra-frequent function words in highly templated narratives. TF-IDF is a strong, inexpensive competitor here because the narratives are structured English with repeated lexical markers.

LLM evaluator. The LLM baseline reads each narrative through the OpenAI Chat Completions API (default model `gpt-5-mini-2025-08-07`) and returns a structured JSON object with `risk_score`, `risk_level`, `primary_indicators`, `explanation`, and `recommended_action`. Each sample is sent as a two-message request of the form `messages=[system, user]`; the user message may include optional CALIBRATION EXAMPLES followed by the real narrative bracketed by the literal markers `NARRATIVE START` and `NARRATIVE END`. The prompt forbids invention of entities and requires the explanation to cite only facts present in the narrative; the model must return exactly one JSON object and no Markdown code fence. Temperature and other generation settings follow the defaults in the evaluator script. Exact

prompt templates are fixed strings in `llm_evaluator.py` to support replication.

Two prompt styles are compared. The chain-of-thought (CoT) prompt instructs the model to reason internally through six ordered steps—*Entities*, *Relations*, *Benign hypothesis*, *Threat hypothesis*, *Evidence checklist*, and *Score*—before mapping the result to the shared scoring rubric. The minimal prompt omits this six-step scaffold. The anti-hallucination constraints, JSON schema, and scoring rubric are identical across styles, so the two prompts differ only in whether the explicit reasoning scaffold is present.

Fusion model. The repository supports a calibrated fusion stage that combines standardized score columns through logistic regression. The exact feature stack is configuration-dependent: common runs combine LLM and TF-IDF scores, while the utility can also incorporate additional inputs such as the Stage 1 score unless explicitly disabled. Fusion numbers in the paper should therefore be read as the best-performing configured score stack for the reported run, not as a universal two-input default.

E. Novelty and contributions

The individual components of the pipeline are deliberately standard. The contribution lies instead in their organization and evaluation: a practical two-stage architecture, a narrative representation that carries relational context into Stage 2, and a comparison protocol strong enough to support the later discussion of what each component actually contributes. The detailed interpretation of those contributions is deferred to the Results and Discussion sections rather than repeated here.

IV. DATASET AND FEATURE SPACE

A. Dataset

The experiments are built around the CERT r4.2 insider-threat dataset, a synthetic but widely used benchmark designed to support controlled insider-threat research [7]. The repository uses official ground truth from `answers/insiders.csv`. In this study, the dataset filter is 4.2, which yields 70 insider records associated with 70 unique malicious users. Those users belong to three scenario families: 30 scenario-1 cases, 30 scenario-2 cases, and 10 scenario-3 cases. The scenarios describe behaviors such as after-hours work with removable media, job-search and competitor contact before departure, and disgruntled administrator misuse.

The sample unit used for learning is not a raw event but a user-day window. Preprocessing produces approximately 330,285 user-day windows, of which 1,364 are malicious. This corresponds to a malicious rate of about 0.413%, which makes the task extremely imbalanced and explains why PR-AUC and recall-oriented evaluation are more informative than plain accuracy.

The data sources used by the pipeline are broader than the Stage 1 feature table alone. Stage 1 uses logon, file, device, and email logs. Stage 2 enriches shortlisted windows with HTTP and LDAP information to build narratives. This separation is intentional: it keeps the first stage cheap while reserving

richer context for the much smaller set of cases that survive screening.

B. Feature engineering

Stage 1 uses 17 structured features. Sixteen are produced directly during user-day aggregation, and one additional interaction feature is added in the screening script. Table I groups these features by source and purpose.

Reproducibility note. In the released pipeline, these 17 user-day features come from the multi-source daily aggregation path that writes the Stage 1 table used by screening (e.g., `features.csv`). This is the feature path assumed throughout the experiments reported here; it should not be confused with other repository utilities that construct different event-level or differently named feature tables for exploratory purposes.

Feature construction note. The aggregated user-day table already includes `logon_after_hours_ratio`, defined as the fraction of logons within the window that occur outside nominal business hours. The screening script additionally creates `off_hour_activity_ratio`. In the default `features.csv` path, per-event `is_after_hours` flags are not retained, so `off_hour_activity_ratio` is populated by copying `logon_after_hours_ratio`—that is, in the reported pipeline it acts as a backward-compatibility alias rather than an independent signal. The separate column name is retained so the released screening code remains compatible with earlier interfaces and with future extensions in which richer row-level timestamp features could support a more general activity-based off-hours ratio.

These features were selected to provide just enough information for first-pass screening. Instead of processing every raw event, the first stage captures volume, sensitivity, timing, and deviation from personal baseline. Count features summarize how much happened; sensitivity-oriented features capture potentially valuable files and unusual outbound email; time-based features highlight after-hours behavior. The `*_vs_user_avg` terms measure sudden amplification relative to the user’s own normal pattern, which is critical because insider threats often look more like deviations from personal baseline than deviations from organizational average.

C. Descriptive statistics

Descriptive statistics are reported over the Stage 1 suspicious pool of 9,908 windows forwarded to Stage 2, which is the data partition directly used for downstream model training and evaluation. The pool spans 2010-01-02 to 2011-05-16 and includes 1,224 malicious windows from 145 unique users, corresponding to a malicious rate of 12.35%. This is substantially higher than the 0.41% rate in the full dataset, confirming that Stage 1 screening concentrates suspicious activity. Table II summarizes key feature statistics for this pool.

An analysis of these statistics highlights a clear behavioral pattern. The medians for several activity counts are small, but the maxima and relative-to-history ratios are large, indicating a heavy-tailed distribution in which a minority of windows

TABLE I
STAGE 1 FEATURE SPACE USED FOR USER-DAY SCREENING.

Category	Features
Logon activity	n_logon, n_logon_after_hours, logon_after_hours_ratio
File activity	n_file, n_file_sensitive, n_file_sensitive_dir
Device activity	n_device_connect, n_device_disconnect, n_device_total
Email activity	n_email, n_email_external, n_email_abnormal_attachment
Burst relative to user history	n_logon_vs_user_avg, n_file_vs_user_avg, n_device_total_vs_user_avg, n_email_vs_user_avg
Additional interaction	off_hour_activity_ratio

TABLE II
REPRESENTATIVE DESCRIPTIVE STATISTICS FOR THE ARCHIVED XGBOOST SUSPICIOUS POOL (9,908 WINDOWS).

Feature	Mean	Median	Std.	Max
n_logon	1.131	1.000	0.466	8.000
n_file	1.369	0.000	2.135	27.000
n_device_total	2.777	2.000	2.923	28.000
n_email	7.000	9.000	4.894	18.000
n_file_sensitive	1.145	0.000	1.826	23.000
n_email_external	2.849	2.000	3.088	14.000
n_email_abnormal_attachment	0.462	0.000	0.742	6.000
n_file_vs_user_avg	1.191	0.000	9.330	334.888
n_device_total_vs_user_avg	1.300	0.492	7.945	334.972
risk_score	0.687	0.689	0.202	1.000

show very sharp spikes. This is exactly what one would expect in insider screening: most shortlisted windows remain close to everyday behavior, while a smaller subset contains abrupt bursts of device, file, or email activity. The saved malicious windows are also scenario-skewed: 1,089 of the 1,224 positives belong to scenario 2, while only 12 belong to scenario 3. This imbalance becomes important later when interpreting why some models perform better than others.

V. EXPERIMENTAL METHODOLOGY

A. Hyperparameter selection

Stage 1 uses user-level splitting to avoid leakage: the same user is never allowed to appear in both train and test partitions. In the main experiment the split yields 800 train users and 200 test users, corresponding to 265,208 training windows (0.45% malicious) and 65,077 test windows (0.28% malicious).

Hyperparameters are selected through RandomizedSearchCV with recall as the optimization objective.

- **XGBoost search space:** $n_estimators \in \{80, 120, 200\}$, $max_depth \in \{3, 4, 6\}$, $learning_rate \in \{0.05, 0.08\}$, $subsample \in \{0.75, 0.85\}$, $colsample_bytree \in \{0.75, 0.85\}$, $min_child_weight \in \{2, 4\}$, $gamma \in \{0.1, 0.2\}$, $reg_alpha \in \{0.1, 0.2\}$, $reg_lambda \in \{1.0, 2.0\}$.
- **Random Forest search space:** $n_estimators \in \{80, 120, 200\}$, $max_depth \in \{8, 12\}$, $min_samples_split \in \{8, 16\}$, $min_samples_leaf \in \{4, 8\}$.

The best saved XGBoost configuration uses $n_estimators = 200$, $max_depth = 6$, $learning_rate = 0.08$, $subsample =$

0.75 , $colsample_bytree = 0.85$, $min_child_weight = 2$, $\gamma = 0.2$, $reg_alpha = 0.2$, and $reg_lambda = 2.0$; the best Random Forest configuration uses $n_estimators = 120$, $max_depth = 12$, $min_samples_split = 16$, and $min_samples_leaf = 4$.

Stage 1 supports cost-sensitive weighting and optional SMOTE/ADASYN over-sampling; the main run uses SMOTE. The user-level 80/20 split is performed *before* over-sampling, so held-out test users never receive synthetic samples. Inside the randomized search, however, SMOTE is applied once on the full training partition before internal fold creation rather than separately within each fold’s training subset. Consequently, the internal cross-validation scores used for model selection may be mildly optimistic; this does not affect the final held-out test metrics, which remain user-disjoint. The Stage 1 decision threshold is selected from training predictions to maximize recall subject to a minimum-recall constraint.

Stage 2 is intentionally lighter. The rule baseline has no learned hyperparameters. The TF-IDF baseline uses the configuration already described in Section III-D. The LLM configuration follows Section III-D. The fusion model trains logistic regression on standardized score inputs from the designated training split; the exact score stack is determined by the command-line configuration used for a given run.

B. Evaluation metrics and testing protocol

The metrics are chosen to match the operational goals of each stage rather than to satisfy a generic machine-learning checklist.

a) *Stage 1 metrics.*: Precision, recall, F1, and PR-AUC are reported on a held-out user split. PR-AUC is especially appropriate under extreme class imbalance because it measures ranking quality in the regime where false positives are abundant. In addition, Top- K recall and Top- K precision are reported because Stage 1 is fundamentally a ranking-and-filtering module: these metrics answer the analyst’s question of how much malicious activity remains reachable if only the top 3% or 5% of windows can be reviewed downstream.

The paper therefore uses two distinct families of Stage 1 metrics. Test precision, recall, F1, and PR-AUC are computed only on held-out test users and measure ranking quality on unseen users. Top- K recall, Top- K precision, and candidate-pool size are computed by ranking *all* scored user-day windows and taking the global top- K percent; they answer a deployment-

style question about full-population screening under a budget constraint. Reporting both views is intentional: the former emphasizes user-level generalization, the latter emphasizes pool-level coverage.

b) Stage 1 validation.: In addition to the hold-out split, the project runs 5-fold user-level cross-validation using `GroupKFold`. This path does *not* apply SMOTE even if over-sampling is enabled elsewhere, and it uses the fixed model-training routines provided by the evaluation entry point rather than re-running the main randomized hyperparameter search inside each fold. It is therefore a lighter, separate protocol that remains free of the optimism caused by applying SMOTE before internal cross-validation. Because the two paths differ in this way, readers should not mix numbers from the main 80/20 search and the 5-fold user-level evaluation without first checking which configuration produced them.

c) Stage 2 metrics.: Precision, recall, F1, and PR-AUC are reported on an aligned evaluation split within the suspicious pool. The main tables use a best-F1 threshold sweep on the evaluation set for comparability across methods. The released utilities also support fixed-recall, fixed-precision, and fixed-threshold operating points; deployment-oriented operating points should be read from those modes, with thresholds selected on a dedicated validation split rather than on the final evaluation partition. The main F1-sweep protocol is useful for method comparison but is slightly optimistic from a deployment perspective because the evaluation set itself participates in threshold selection. This limitation is revisited in the discussion.

d) Scenario skew caveat.: The Stage 1 shortlist is heavily skewed toward one scenario family in CERT r4.2: 1,089 of the 1,224 retained positives belong to scenario 2, while scenario 3 contributes only 12 windows. Aggregate Stage 2 metrics therefore primarily reflect scenario-2 coverage, and any future per-scenario stratification would need to be interpreted cautiously because the positive support for scenarios 1 and 3 is too sparse for stable precision/recall estimates on this snapshot.

e) Temporal-slice robustness.: `eval_robustness_time_quartiles.py` partitions evaluation windows by calendar date into quartiles and recomputes ranking and classification metrics within each slice using fixed model scores and no retraining. This is not a full domain-adaptation study but a lightweight diagnostic for whether narrative-based scores remain stable across earlier and later portions of the benchmark. Large quartile gaps would signal temporal sensitivity and therefore weaker robustness than the aggregate evaluation alone might imply.

f) Stage 2 testing protocol.: The TF-IDF baseline defaults to `GroupShuffleSplit` by user with an 80/20 split, and the LLM is restricted to the same train/test keys via JSON so that all narrative scorers are evaluated on identical windows. This protocol preserves fair method comparison while avoiding the prohibitive cost of repeated full LLM cross-validation.

C. Baseline and competitor methods

Strong baselines are essential here because the proposed method is hybrid. Without serious competitors, any improve-

ment would be hard to interpret. The project therefore evaluates the following methods:

- **Random Forest Stage 1 competitor.** Tests whether XGBoost is genuinely the better screener or merely the default choice.
- **Stage 1 pool-only baseline.** Escalates every shortlisted window; answers whether Stage 2 adds value at all.
- **Rule baseline.** An interpretable, zero-training reference point.
- **TF-IDF + Logistic Regression.** The strongest non-LLM text baseline and an especially relevant competitor because the Stage 2 representation is textual.
- **LLM baseline.** Measures whether semantic reasoning over graph-informed narratives adds value beyond lexical matching.
- **Fusion baseline.** Tests whether different second-stage signals are complementary rather than mutually exclusive.

Each represents a defensible alternative a reader would reasonably ask about. If the LLM wins, it must beat TF-IDF; if the hierarchy wins, it must beat the naive strategy of escalating every shortlisted window; if XGBoost wins, it must beat another strong tree-based model. This makes the conclusions more credible. The main Stage 1 cross-validation summary appears in Table III; the next section interprets it together with the separate held-out test results so that the two evaluation paths are not conflated.

VI. RESULTS

A. Stage 1 screening results

This section reports Stage 1 results in three parts: the separate 5-fold user-level cross-validation path, the main held-out 80/20 test path, and the Top- K screening analysis used to set the Stage 2 workload budget. Table III reports the 5-fold cross-validation summary. XGBoost and Random Forest deliver similarly high mean recall across folds, but XGBoost achieves substantially higher PR-AUC, indicating better ranking quality under imbalance. Random Forest attains a notably higher mean F1 in cross-validation, but this advantage disappears on the main held-out test split where XGBoost is stronger on every key operational metric.

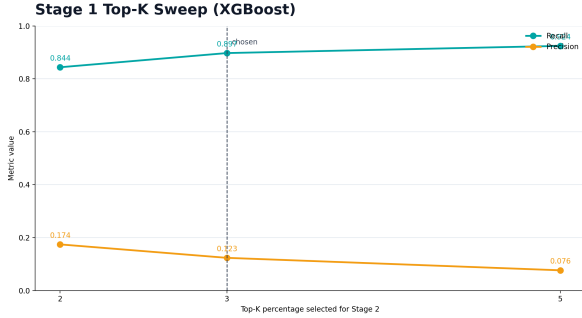
The held-out test and Top-3% results are shown in Table IV. XGBoost is the better Stage 1 model for this project. On held-out test users, it achieves a higher test recall (0.4863 vs. 0.4590), higher test F1 (0.1710 vs. 0.1323), and higher PR-AUC (0.2837 vs. 0.1963). Under the global ranking view, it also achieves stronger Top-3% Recall@Pool (0.8974 vs. 0.8622). In practical terms, the XGBoost screener keeps nearly 90% of all malicious windows available to Stage 2 while forwarding only 9,908 windows out of 330,285.

The Top- K sweep in Figure 1 makes the trade-off visually clear. Reducing K from 5% to 2% increases precision but drops recall. The rationale for treating $K = 3\%$ as the main operating point is given in Section III-B; in short, 3% provides the reported compromise between Stage 2 workload and Recall@Pool, with a candidate pool of roughly ten thousand windows and 89.74% XGBoost Recall@Pool.

TABLE III

STAGE 1 USER-LEVEL 5-FOLD CROSS-VALIDATION RESULTS (MEAN $\pm$ STD) UNDER THE SEPARATE FIXED-HYPERPARAMETER EVALUATION PATH WITHOUT SMOTE; THIS TABLE IS NOT PRODUCED BY THE MAIN 80/20 + HYPERPARAMETER-SEARCH PIPELINE.

Model	Precision	Recall	F1	PR-AUC
XGBoost	0.0259 $\pm$ 0.0081	0.8998 $\pm$ 0.0790	0.0501 $\pm$ 0.0153	0.3569 $\pm$ 0.0492
Random Forest	0.0375 $\pm$ 0.0122	0.8924 $\pm$ 0.0829	0.0715 $\pm$ 0.0221	0.2486 $\pm$ 0.0459

Fig. 1. Top- K screening trade-off for the Stage 1 XGBoost model.

Feature importance analysis helps explain these results. In the reported XGBoost experiment, the most important feature is `n_device_disconnect`, followed by `n_device_connect`, `n_logon`, `n_device_total_vs_user_avg`, and `n_logon_vs_user_avg`. This ordering is consistent with the insider scenarios embedded in CERT r4.2, which emphasize removable-device exfiltration and abrupt deviations from personal baseline.

B. Stage 2 results

The Stage 2 results on the aligned evaluation split are reported in Table V. The first finding is immediate: the Stage 1 pool-only baseline has perfect recall but poor precision (0.129). This confirms that Stage 1 alone is not enough—a second-stage filter is necessary if the goal is to reduce analyst burden rather than simply to gather a shortlist.

The second finding is more surprising. The strongest single Stage 2 model is not the LLM; it is TF-IDF with logistic regression, which achieves precision 0.315, recall 0.787, F1 0.450, and PR-AUC 0.365. The LLM with chain-of-thought prompting reaches F1 0.323, numerically identical to the rule baseline, suggesting that at this operating point the LLM adds no discriminative advantage over hand-crafted keyword rules—and both remain notably below TF-IDF. A plausible explanation is that the current narratives are semantically meaningful but partially templated, which makes repeated lexical markers particularly easy for TF-IDF to exploit. The best reported fusion run improves recall to 0.810 and attains F1 0.432. This does not surpass the TF-IDF baseline in F1, but it demonstrates complementarity: combining multiple score sources can add useful signal even when no single semantic model is the best standalone classifier on this split.

Stage 2 Main Results on the Aligned Test Split

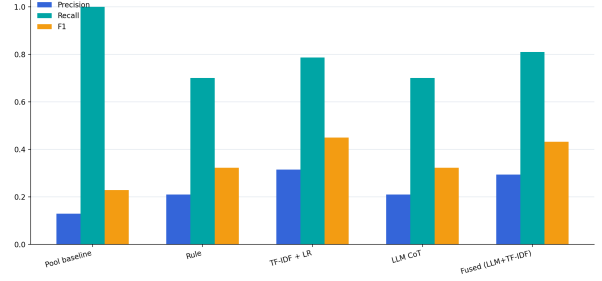


Fig. 2. Stage 2 comparison across pool-only, rule, TF-IDF, LLM, and fused methods.

The prompt ablation also reveals an important detail. The chain-of-thought and minimal prompts produce the same F1 (0.323), with CoT only slightly improving recall (0.700 vs. 0.677). This suggests that the current bottleneck is not primarily prompt sophistication; the main limit more likely lies in the information structure of the narratives themselves or in their relatively templated form.

VII. CRITICAL DISCUSSION OF THE RESULTS

Why did the reported results take this shape? The answer becomes clearer once one considers both the benchmark and the representation.

First, Stage 1 works well because the CERT r4.2 scenarios contain strong, repeated activity patterns that are visible in aggregated user-day features. Removable-device behavior, abrupt spikes relative to user history, and off-hours activity are exactly the signals a tree-based model can rank effectively. That is why XGBoost can recover nearly 90% of malicious windows in only 3% of the search space.

Second, the Stage 2 findings show that richer representation does not automatically imply that a larger model will win. The narratives are structured English generated from templates plus grounded facts. That structure is ideal for TF-IDF because repeated lexical cues such as “USB device,” “external email,” “after-hours,” and “no prior access observed” are highly predictive. In other words, the narrative format itself makes lexical modeling unusually competitive, which explains why TF-IDF outperforms the LLM as a standalone classifier.

Third, the LLM’s value appears more qualitative and complementary than dominant. It produces readable rationales and can combine multiple weak clues into a coherent judgment, but on this benchmark and this representation it does not outperform the simpler text baseline. The fusion result is therefore revealing: the LLM adds signal that improves recall when combined with TF-IDF, yet not enough to surpass TF-IDF’s own best F1. This is a more nuanced and more realistic outcome than a blanket claim that the LLM is best.

Fourth, the benchmark itself likely shapes the outcome. The saved XGBoost suspicious pool is heavily dominated by scenario-2 positives. Such skew favors any method that learns a

TABLE IV

STAGE 1 MAIN HELD-OUT TEST RESULTS AND TOP-3% SCREENING PERFORMANCE FROM THE 80/20 PIPELINE USED FOR MODEL SELECTION AND DOWNSTREAM SCREENING. TEST COLUMNS ARE COMPUTED ON HELD-OUT USERS ONLY; TOP-3% COLUMNS ARE COMPUTED FROM A GLOBAL RANKING OVER ALL SCORED WINDOWS.

Model	Test P	Test R	Test F1	PR-AUC	Top-3% R	Top-3% P
XGBoost	0.1037	0.4863	0.1710	0.2837	0.8974	0.1235
Random Forest	0.0773	0.4590	0.1323	0.1963	0.8622	0.1187

TABLE V

STAGE 2 MAIN RESULTS ON THE ALIGNED EVALUATION SPLIT.

Method	Precision	Recall	F1	PR-AUC
Stage 1 pool only	0.129	1.000	0.229	N/A
Rule baseline	0.210	0.700	0.323	0.185
TF-IDF + Logistic Regression	0.315	0.787	0.450	0.365
LLM (CoT prompt)	0.210	0.700	0.323	0.191
Best fusion run	0.294	0.810	0.432	0.317

strong lexical or behavioral template for that scenario. Scenario 3, by contrast, is represented by only 12 malicious windows in the saved pool, which makes it harder to assess whether the second-stage methods generalize equally well across threat types.

Finally, some methodological choices make the reported results slightly optimistic and should be interpreted carefully. Stage 2 thresholds are selected by sweeping the evaluation set itself for best F1, which is useful for fair method comparison but not equivalent to deployment-time calibration on a separate validation set. Likewise, the repository snapshot does not include every derived artifact, which limits how much full-dataset descriptive analysis can be reproduced from versioned files alone.

VIII. RELEVANCE OF THE RESULTS, ADVANTAGES, LIMITATIONS, AND LESSONS LEARNED

The study achieves a clear practical result: a hierarchical insider-threat pipeline can preserve high recall while sharply shrinking downstream workload. This is the most relevant operational contribution. Instead of forcing a security team to inspect more than 330,000 user-day windows, the Stage 1 screener reduces the candidate set to fewer than 10,000 windows while still retaining almost 90% of malicious cases.

The results also support a more nuanced scientific claim. They do *not* support the simplistic argument that LLMs should replace classical models in insider-threat detection. What they support is a more defensible statement: graph-informed narrative representations are useful, but their benefits depend on how the second-stage model consumes them. In this project, TF-IDF is the strongest single narrative scorer, while LLMs become more useful when treated as one source of signal among several rather than as a monolithic final judge.

Several advantages emerge from the study:

- The system is modular which ensures Stage 1 and Stage 2 can be improved independently.

- The method is cost-aware because expensive reasoning is applied only to a shortlist.
- The narratives are interpretable, allowing a human analyst to read the same representation used by the models.
- The evaluation includes credible baselines, which makes the conclusions more trustworthy.

The limitations are equally important:

- CERT r4.2 is synthetic, so ecological validity remains limited.
- The suspicious pool is scenario-skewed, which may distort second-stage comparisons.
- Stage 2 evaluation uses threshold selection on the evaluation set, which is convenient but optimistic.
- When SMOTE is enabled for Stage 1 model selection, over-sampling is applied once on the full training partition before internal cross-validation rather than separately within each fold, so the internal CV scores may be optimistic, even though the final held-out test split remains user-disjoint.
- The project’s “graph” is an implicit relational narrative rather than a fully explicit graph-learning system.
- Some generated data files are excluded from version control, so not every descriptive statistic is archived directly in the repository snapshot.

Ultimately, the primary contribution of this study extends beyond specific performance metrics. It underscores the methodological lesson that strong, well-constructed classical baselines remain indispensable even in LLM-era cybersecurity research. Another lesson is that the best place for an LLM may be *after* careful problem compression, not before it. The experiments also show that insider detection should be evaluated as an operational pipeline, not only as a flat classification problem.

IX. CONCLUSION

This project presents and evaluates a two-stage insider-threat detection pipeline in which Stage 1 performs inexpensive Top- K screening and Stage 2 applies richer narrative-based scoring only to shortlisted windows. As the Results and Critical Discussion sections show, the hierarchy is operationally useful: XGBoost is the strongest Stage 1 screener in this repository, TF-IDF is the strongest single Stage 2 model on the aligned split, and the best reported fusion run improves recall through partial complementarity. The broader lesson is not that one fashionable model class wins universally, but that effective insider-threat detection depends on matching

the representation and the model to the role each stage plays. Readers interested in the practical takeaway should remember three points: hierarchical screening makes the problem tractable, strong non-LLM baselines remain competitive against narrative-consuming LLMs when the narratives are partially templated, and fusion provides a pragmatic path to combining semantic and lexical evidence without forcing a single-model winner.

X. FUTURE WORK

Several next experiments would be worthwhile and would strengthen the study substantially.

- **Run a same-budget random-pool baseline.** A decisive follow-up is to compare the current Stage 1 shortlist against a random shortlist of the same size before applying the LLM. This would quantify how much the hierarchy itself contributes beyond narrative reasoning.
- **Complete and report the full narrative ablation.** The repository already supports removing LDAP context, historical comparison, and cross-source chronology. Reporting those ablations cleanly would show which contextual components actually drive gains.
- **Calibrate Stage 2 on a dedicated validation split.** Thresholds should be selected without looking at the final evaluation set to make the second-stage results more deployment-realistic.
- **Test explicit graph representations.** The present system verbalizes relationships into text; a natural extension is to compare this design against explicit graph construction with graph neural networks or graph-based anomaly scoring.
- **Reduce narrative template rigidity.** If the narratives become less templated and more semantically varied, the balance between TF-IDF and LLM scoring may change.
- **Evaluate on additional benchmarks.** Extending the study to CERT v6.2 or other insider datasets would test whether the current findings generalize beyond r4.2.
- **Report richer operating-characteristic summaries.** Deployment-facing cost curves, workload-vs.-recall plots, and threshold-calibration summaries on a dedicated validation split would allow operating points to be compared without relying only on best-F1 sweeps.

REFERENCES

- [1] I. A. Gheyas and A. E. Abdallah, "Detection and prediction of insider threats to cyber security: A systematic literature review and meta-analysis," *Big Data Analytics*, vol. 1, no. 1, Art. no. 6, 2016, doi: 10.1186/s41044-016-0006-0.
- [2] I. Homoliak, F. Toffalini, J. Guarnizo, Y. Elovici, and M. Ochoa, "Insight into insiders and IT: A survey of insider threat taxonomies, analysis, modeling, and countermeasures," *ACM Computing Surveys*, vol. 52, no. 2, Art. no. 30, 2020, doi: 10.1145/3303771.
- [3] S. Yuan and X. Wu, "Deep learning for insider threat detection: Review, challenges and opportunities," *Computers & Security*, vol. 104, Art. no. 102221, May 2021, doi: 10.1016/j.cose.2021.102221.
- [4] W. Eberle and L. Holder, "Insider threat detection using graph-based approaches," *J. Appl. Secur. Res.*, vol. 6, no. 1, pp. 32–81, Dec. 2010, doi: 10.1080/19361610.2011.529413.
- [5] H. Eldardiry, E. Bart, J. Liu, J. Hanley, B. Price, and O. Brdiczka, "Multi-domain information fusion for insider threat detection," in *Proc. 2013 IEEE Security and Privacy Workshops (SPW)*, 2013, pp. 45–51, doi: 10.1109/SPW.2013.14.
- [6] A. R. Tuor, S. P. Kaplan, B. J. Hutchinson, N. M. Nichols, and S. M. Robinson, "Deep learning for unsupervised insider threat detection in structured cybersecurity data streams," in *Proc. AAAI Workshop on Artificial Intelligence for Cyber Security*, 2017, pp. 224–231.
- [7] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *Proc. 2013 IEEE Security and Privacy Workshops (SPW)*, 2013, pp. 98–104, doi: 10.1109/SPW.2013.37.
- [8] W. Hong, J. Yin, M. You, H. Wang, J. Cao, J. Li, M. Liu, and C. Man, "A graph empowered insider threat detection framework based on daily activities," *ISA Trans.*, vol. 141, pp. 84–92, Oct. 2023, doi: 10.1016/j.isatra.2023.06.030.
- [9] Y. Chen, M. Cui, D. Wang, Y. Cao, P. Yang, B. Jiang, Z. Lu, and B. Liu, "A survey of large language models for cyber threat detection," *Computers & Security*, vol. 145, Art. no. 104016, Oct. 2024, doi: 10.1016/j.cose.2024.104016.
- [10] S. Song, Y. Zhang, and N. Gao, "Confront insider threat: Precise anomaly detection in behavior logs based on LLM fine-tuning," in *Proc. 31st Int. Conf. Comput. Linguistics (COLING)*, 2025, pp. 8589–8601. [Online]. Available: <https://aclanthology.org/2025.coling-main.574/>